CS509 - Assignment 2

Purpose. This document defines the algorithms, text-file input formats, expected outputs, and timing requirements for Assignment 2. The driver program must read the input file, prepare the required data structure, call the algorithm, and print both the final result and the algorithm execution time.

Timing rule. Measure only the algorithm execution time. File reading, input parsing, adjacency-list-to-CSR conversion, output printing, and other setup work must not be included.

1. Assignment Structure

Task Type	Algorithms	Work Mode
Individual task	Bellman-Ford (single-source shortest path on graph with possible negative edge weights) and Floyd-Warshall (all-pairs shortest path), CSR graph as input	Individual
Buddy task	Triangle Counting, Betweenness Centrality and Connected Components on undirected graphs	Pair of two students

2. Brief Explanation of the Algorithms

2.1 Bellman-Ford (link)

Bellman-Ford computes the shortest distance from a single source vertex to every other reachable vertex, and it works correctly even when some edge weights are negative.

- It relaxes every edge in the graph, repeated V-1 times, where V is the number of vertices.
- After V-1 passes, one additional pass is done: if any edge can still be relaxed, a negative-weight cycle is reachable from the source and no shortest path is defined for the affected vertices. <check the meaning, clarify>
- Running time is O(V·E), which is slower than Dijkstra's algorithm but is required whenever negative edge weights are present.

2.2 Floyd-Warshall (link)

Floyd-Warshall computes the shortest distance between every pair of vertices in the graph in a single run.

- Brute-force approach.
- dynamic-programming algorithm approach. For every intermediate vertex k, it updates dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]) for every pair (i, j).
    - It also tolerates negative edge weights, provided the graph has no negative-weight cycle.

- Running time is $O(V^3)$ and memory usage is $O(V^2)$, since the full distance matrix is kept in memory. Because of this, Floyd-Warshall is only required on the smaller graph sizes listed in Section 4.2.

2.3 CSR Graph

CSR (Compressed Sparse Row) is a compact format used to store sparse graphs efficiently. For a graph, the adjacency list is converted into three arrays:

- **row_ptr**: tells where each vertex's neighbour list starts and ends.
- **col_idx**: stores the neighbouring vertex numbers.
- **values**: stores edge weights, including negative weights where the algorithm allows them.

CSR applies to Bellman-Ford, Triangle Counting, Betweenness Centrality and Connected Components, since all four read an adjacency list.

2.4 Triangle Counting (link)

A triangle is a set of three vertices that are pairwise connected by edges in an undirected graph. Triangle counting reports how many such sets exist.

- A common approach is: for every vertex u, examine each pair of neighbours of u using the CSR, and check whether that pair is itself connected by an edge, i.e., common-neighbours.
- Each triangle is discovered once at each of its three vertices, so the raw count must be divided by 3 to get the final answer.
- A suggested optimization to speedup: Sorting each vertex's neighbour list first allows the common-neighbour check to run faster than a linear scan.

USE CASES:

• Detecting tightly connected friend groups in social networks (Facebook, LinkedIn).

• Discovering communities in collaboration or citation networks.

• Fraud detection by identifying suspicious groups of mutually connected accounts.

• Biological network analysis (protein interaction networks).

• Measuring the clustering coefficient of a graph.

2.5 Betweenness Centrality (link)

Betweenness centrality measures how often a vertex lies on the shortest paths between other pairs of vertices. A vertex with high betweenness centrality acts as a bridge or bottleneck in the graph. **<use cases>**

- For unweighted graphs, Brandes' algorithm is the required approach. It runs a BFS (you can call your BFS code from Assignment 1) from every vertex and accumulates, for each vertex v, the fraction of shortest paths between other pairs that pass through v.
- Report the raw (unnormalized) centrality value for every vertex, i.e. do not divide by (V-1)(V-2).

- Running time is O(V·E), so Betweenness Centrality also uses the reduced graph sizes described in Section 4.2 below.

USE CASES:

• Identifying influential people in social networks.
• Finding critical routers or servers whose failure would affect communication.
• Road traffic planning by locating important intersections.
• Epidemic spread analysis to identify people who can spread diseases most effectively.
• Detecting bottlenecks in transportation, logistics, or communication networks.

2.6 Connected Components (link)

A connected component is a maximal set of vertices in an undirected graph such that a path exists between every pair of vertices in the set.

- Connected components can be found with repeated BFS or DFS, starting a new traversal from any unvisited vertex, or with a Union-Find (Disjoint Set) structure built while scanning the edge list. You are free to call BFS/DFS codes from Assignment 1.
- Every vertex, including an isolated vertex with no edges, must be assigned to exactly one component.

USE CASES:

• Detecting isolated communities in social networks.

• Identifying disconnected subnetworks in computer networks.

• Image processing (connected component labeling).

• Grouping related webpages or documents into disconnected clusters.

• Finding isolated regions in GIS (Geographic Information Systems).

3. General Input-File Rules

- Every test case must be stored in a separate .txt file.
- Use space-separated integer values unless otherwise stated. Edge weights for Bellman-Ford and Floyd-Warshall may be negative integers.
- Vertex numbering must be consistent throughout the semester. The recommended convention is 0 to V-1.
- Bellman-Ford, Triangle Counting, Betweenness Centrality and Connected Components use an adjacency-list input file.
- Bellman-Ford graphs are directed. Triangle Counting, Betweenness Centrality and Connected Components graphs are undirected, so each edge must appear in the adjacency list of both endpoint vertices.

- Do not place a negative weight on an undirected edge: travelling back and forth across it would form a negative-weight cycle by itself. Negative weights are only meaningful on directed edges.
- The required graph sizes differ per algorithm because of their time and memory complexity; see Section 4.2 for the exact list.
- You are expected to call the CSR conversion function from the previous assignment. DO NOT copy the code into this assignment.

4. CSR Graph

4.1 Graph Input Format and CSR Conversion

- **The input format for Bellman-Ford, Triangle Counting, Betweenness Centrality and Connected Components is an adjacency list.**
- A helper function must be provided to convert the adjacency-list representation into Compressed Sparse Row (CSR) format before the algorithm runs.
- The adjacency-list-to-CSR conversion is preprocessing. Its execution time must not be included in the reported algorithm runtime.
- For an algorithm that uses CSR, timing must begin only after the CSR representation has been prepared.
- **Floyd-Warshall is exempt from CSR conversion**; its dense matrix is read directly from the input file (Section 6.1), and timing begins after the matrix has been loaded into memory.

4.2 Required Graph Input Sizes

Graph sizes are not the same for every algorithm in this assignment. Floyd-Warshall's $O(V^3)$ time and $O(V^2)$ memory make the standard 50,000 / 100,000 scale infeasible, and Betweenness Centrality's $O(V \cdot E)$ cost via Brandes' algorithm makes the two largest standard sizes impractical as well, so both use a reduced scale. Use the sizes below for each algorithm:

Algorithm	Required Number of Vertices (V)	Note
Bellman-Ford	10, 100, 10,000, 50,000, 100,000	Keep the two largest graphs sparse ($E \approx 2V$ to $4V$) so that $O(V \cdot E)$ stays practical.
Floyd-Warshall	10, 100, 500, 1,000, 2,000	$O(V^3)$ time and $O(V^2)$ memory rule out the 50,000 / 100,000 scale.
Triangle Counting	10, 100, 10,000, 50,000, 100,000	Sort adjacency lists to keep common-neighbour checks fast at scale.
Betweenness Centrality	10, 100, 1,000, 5,000, 10,000	Brandes' algorithm is $O(V \cdot E)$; the two largest sizes are reduced.
Connected Components	10, 100, 10,000, 50,000, 100,000	$O(V+E)$, so the full standard scale applies.

The number of edges (E), graph type (directed/undirected), and other relevant properties must also be recorded for each graph test case in the README.

You can write your own random graph generator file, which takes the number of vertices and edges as input.

In case on your system, for any graph size, execution is not complete (likely core-dump, etc) please mention that in your report.

5. Bellman-Ford Input and Output Test File Format

5.1 Weighted Adjacency-List File Format

The graph is directed. Each neighbour is followed by its edge weight, which may be negative.

```
V E
u0 degree neighbor1 weight1 neighbor2 weight2 ...
u1 degree neighbor1 weight1 neighbor2 weight2 ...
...
u(V-1) degree neighbor1 weight1 neighbor2 weight2 ...
SOURCE s
```

- V: number of vertices. E: number of directed edges.
- For a vertex with no outgoing edges, write: u 0.
- s: source vertex for Bellman-Ford.
- The graph must not contain a negative-weight cycle reachable from s unless the test case is specifically designed to exercise negative-cycle detection (see 5.3).

5.2 Bellman-Ford Example Input

```
5 10
0 2 1 6 3 7
1 3 2 5 3 8 4 -4
2 1 1 -2
3 2 2 -3 4 9
4 2 0 2 2 7
SOURCE 0
```

5.3 Expected Bellman-Ford Output

Print the shortest distance from the source to every vertex, or report that a negative-weight cycle was found.

```
Algorithm: Bellman-Ford
Source: 0
Vertex Distance
0    0
1    2
2    4
3    7
```

```
4     -2
Negative cycle: none
Execution time: <value> ms
```

If the extra relaxation pass finds an edge that can still be relaxed, print "Negative cycle: true" and omit the distance table, since distances are undefined in that case.

6. Floyd-Warshall Input and Output Test File Format

6.1 Adjacency-Matrix File Format

Floyd-Warshall reads a dense V x V matrix instead of an adjacency list. Use INF to mark a pair with no direct edge, and 0 on the diagonal.

```
V
row 0 values
row 1 values
...
row (V-1) values
```

- Each row has exactly V space-separated entries: an integer weight, or the literal token INF.
- Entry (i, j) is the weight of the direct edge from i to j, or INF if no such edge exists. Entry (i, i) must be 0.
- Weights may be negative, but the graph must not contain a negative-weight cycle unless the test case specifically targets negative-cycle detection (see 6.3).

6.2 Floyd-Warshall Example Input

This is the same underlying graph relationship style used in the Bellman-Ford example, restated as a matrix for a 5-vertex graph:

```
5
0 3 8 INF -4
INF 0 INF 1 7
INF 4 0 INF INF
2 INF -5 0 INF
INF INF INF 6 0
```

6.3 Expected Floyd-Warshall Output

Print the full shortest-distance matrix, or report that a negative-weight cycle was found.

```
Algorithm: Floyd-Warshall
Distance matrix:
0  1  -3  2  -4
3  0  -4  1  -1
7  4  0   5  3
```

```
2  -1 -5  0  -2
8  5  1   6  0
Negative cycle: none
Execution time: <value> ms
```

After the algorithm completes, check the diagonal of the resulting matrix. If any dist[i][i] is negative, print "Negative cycle: true" and omit the distance matrix, since a negative-weight cycle makes some distances undefined.

For the graph sizes where both algorithms are required (10 and 100 vertices), run Bellman-Ford from every vertex as source and confirm the resulting distances agree with the corresponding row of the Floyd-Warshall output. Record this cross-check in the README.

7. Triangle Counting Input and Output Test File Format

7.1 Unweighted Undirected Adjacency-List File Format

```
V E
u0 degree neighbor1 neighbor2 ...
u1 degree neighbor1 neighbor2 ...
...
u(V-1) degree neighbor1 neighbor2 ...
```

- V: number of vertices. E: number of undirected edges (count each edge once, even though it appears in two adjacency lists).
- Every edge must appear in the adjacency list of both endpoint vertices.
- For a vertex with no neighbours, write: u 0.
- There is no SOURCE line: triangle counting is computed over the whole graph, not from a single vertex.

7.2 Triangle Counting Example Input

```
6 8
0 2 1 2
1 3 0 2 3
2 3 0 1 3
3 4 1 2 4 5
4 2 3 5
5 2 3 4
```

7.3 Expected Triangle Counting Output

Print the total number of triangles. Listing the individual triangles is required for the two smallest graph sizes and optional above that, since the list itself can become very large.

```
Algorithm: Triangle Counting
```

Total triangles: 3
Triangles found:
(0, 1, 2)
(1, 2, 3)
(3, 4, 5)
Execution time: <value> ms

8. Betweenness Centrality Input and Output Test File Format

8.1 Unweighted Undirected Adjacency-List File Format

Use the same adjacency-list format as Section 7.1. There is no SOURCE line, since centrality is computed for every vertex using all of the other vertices.

8.2 Betweenness Centrality Example Input

5 4
0 1 1
1 2 0 2
2 2 1 3
3 2 2 4
4 1 3

8.3 Expected Betweenness Centrality Output

Print the raw (unnormalized) betweenness centrality value for every vertex, to two decimal places.

Algorithm: Betweenness Centrality
Vertex Centrality
0 0.00
1 3.00
2 4.00
3 3.00
4 0.00
Execution time: <value> ms

9. Connected Components Input and Output Test File Format

9.1 Unweighted Undirected Adjacency-List File Format

Use the same adjacency-list format as Section 7.1, including isolated vertices written as "u 0". There is no SOURCE line.

9.2 Connected Components Example Input

8 4
0 1 1
1 2 0 2
2 2 1 3
3 1 2
4 1 5
5 1 4
6 0
7 0

9.3 Expected Connected Components Output

Print the total number of components and the component id assigned to every vertex. Component ids should be assigned in the order the components are first discovered, starting at 0.

Algorithm: Connected Components
Number of components: 4
Vertex Component
0 0
1 0
2 0
3 0
4 1
5 1
6 2
7 3
Execution time: <value> ms

10. Timing and Measurement Requirements

- Start the timer immediately before calling the algorithm.
- Stop the timer immediately after the algorithm finishes, including the negative-cycle check pass for Bellman-Ford and Floyd-Warshall, since that check is part of the algorithm's defined procedure.
- Do not include file reading, input parsing, memory allocation done as setup, adjacency-list-to-CSR conversion, adjacency-matrix construction, result printing, or file writing.
- Report time in milliseconds or seconds. Use the same unit consistently within a comparison table.
- For very fast inputs, repeated runs may be used and the average may be reported, provided the number of runs is documented.
- For every Bellman-Ford, Floyd-Warshall, Triangle Counting, Betweenness Centrality and Connected Components input file, report the algorithm time for that particular graph.

11. Required README Result Tables

The repository README must contain a completed table for every assignment test case. Add or remove rows as required.

11.1 Bellman-Ford / Floyd-Warshall Results Table

Algorithm	Test File	Vertices	Edges	Source	Negative Cycle	Expected Output	Actual Output	Time	Status
Bellman-Ford	bf_10.txt	10	...	0	No	Distances	...	... ms	Pass/Fail
Floyd-Warshall	fw_10.txt	10	...	N/A	No	Distance matrix	...	... ms	Pass/Fail
...	...	100 / 10000 / 50000 / 100000 (BF) 100 / 500 / 1000 / 2000 (FW)	...	...	...	...	...	...	...

11.2 Graph Analytics Results Table

Algorithm	Test File	Vertices	Edges	Expected Output	Actual Output	Time	Status
Triangle Counting	tc_10.txt	10	...	Total triangles	...	... ms	Pass/Fail
Betweenness Centrality	bc_10.txt	10	...	Centrality per vertex	...	... ms	Pass/Fail
Connected Components	cc_10.txt	10	...	Component per vertex	...	... ms	Pass/Fail
...	...	100 / 10000 / 50000 / 100000 (TC, CC) 100 / 1000 / 5000 / 10000 (BC)	...	...	...	...	...

12. Suggested Test-File Naming

Algorithm	Suggested Names
Bellman-Ford	bf_10.txt, bf_100.txt, bf_10000.txt, bf_50000.txt, bf_100000.txt
Floyd-Warshall	fw_10.txt, fw_100.txt, fw_500.txt, fw_1000.txt, fw_2000.txt

Triangle Counting	tc_10.txt, tc_100.txt, tc_10000.txt, tc_50000.txt, tc_100000.txt
Betweenness Centrality	bc_10.txt, bc_100.txt, bc_1000.txt, bc_5000.txt, bc_10000.txt
Connected Components	cc_10.txt, cc_100.txt, cc_10000.txt, cc_50000.txt, cc_100000.txt

Upload a single file with tables for each algorithm on Google Classroom.

13. Minimum Expected Driver Behaviour

- Accept the selected algorithm and input-file path through the common wrapper/menu or terminal arguments (built in Assignment 1).
- Read and validate the input file, including rejecting a negative weight found on an undirected edge.
- For CSR-based tasks, construct the adjacency list and call the helper function to convert it to CSR.
- Call the selected algorithm.
- Print the final answer in a readable form, or the negative-cycle message where applicable.
- Print only the measured algorithm execution time as the reported timing.
- Return a clear error message for an invalid or missing input file.